Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 9: The Azo Add-on

In this tutorial, we discuss the specific Classes introduced in the **Azo** add-on of **SCOPE**.
It is highly recommended to follow the previous tutorials first.
This tutorial is structured in 4 parts:

1. System and State Subclasses: **System_azo**, **State_azo**
2. Analysis of Thermal Properties.
3. Analysis of Optical Properties: **PSS** and **Lamp** classes
4. Creation of new Azo systems

Introduction to Azo-based photoswitches

Azo-based photoswitches can reversibly interconvert between **trans (E)** and **cis (Z)** isomers.

- **Trans (E)** is usually the thermodynamically more stable form.
- **Cis (Z)** is typically less stable.

Loading [MathJax]/extensions/Safe.js

Light changes the population between these states because each isomer absorbs light differently:

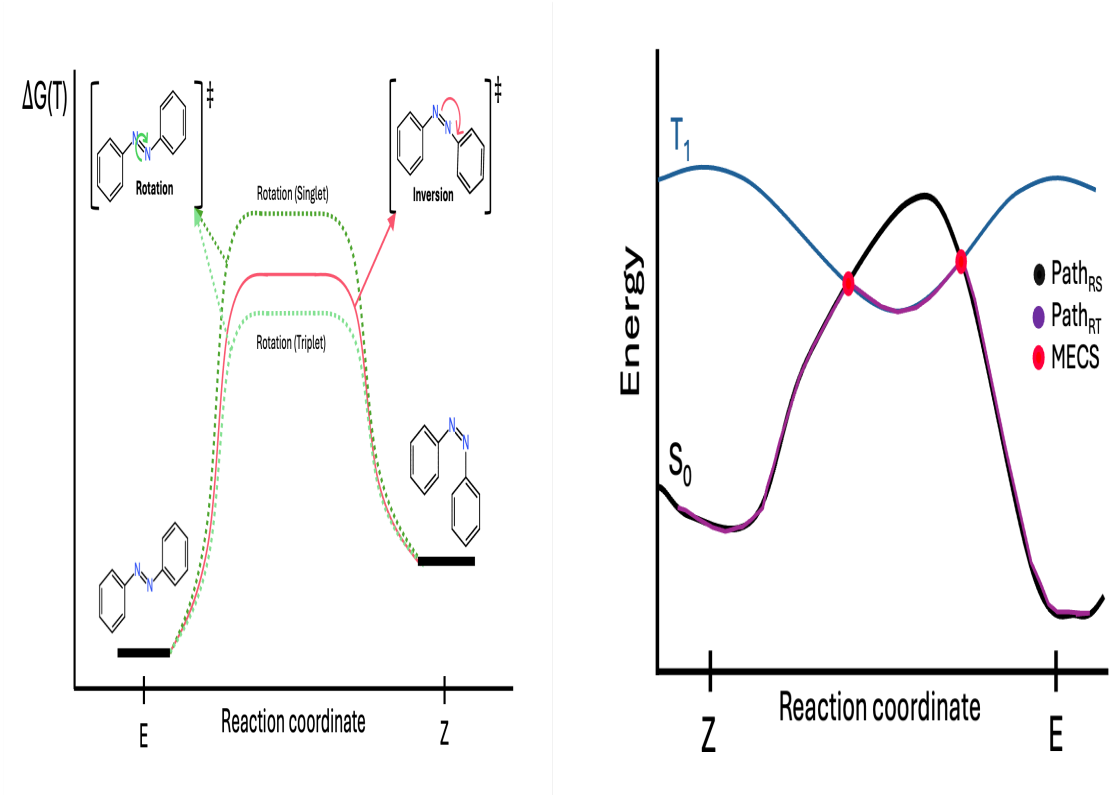
- In the dark, the **trans** isomer is more stable.
- Irradiation at one wavelength enriches **trans -> cis** (photoisomerization).
- Irradiation at another wavelength can drive the **cis -> trans** (back-isomerization).
- Once in **cis**, and without further irradiating, the system gradually reverts to **trans** (thermal relaxation).

Mechanistically, thermal and photo-induced isomerization can proceed through rotation or inversion around the azo unit (N=N) (See **Figure 1**).

- Rotation is characterized by the CNNC dihedral angle around the azo.
- Inversion is characterized by the CNN angles

The energy profiles of these relaxation pathways dictate the thermal stability of the **cis** isomer. To characterize them using Transition State Theory (TST), one needs to find the minima and the transition states (TS) for each pathway.

To make matters worse, there is a "triplet" channel, in which the thermal relaxation proceeds through a rotation-like pathway involving the triplet potential energy surface (PES). This channel involves two **inter-system crossing (ISC)** steps: S-to-T, T-to-S. It is the energy of those crossing points (and not that of the triplet minimum) what matters in the context of TST.



Loading [MathJax]/extensions/Safe.js

Figure 1. Free energy diagram along the reaction coordinate. E and Z correspond to the trans and cis isomers, respectively. The solid red line represents the inversion pathway, while the dark and light green dotted lines represent the rotation pathways on the singlet and triplet potential energy surfaces (PES), respectively.

Figure 2. Description of the two rotation pathways in the photo-conversion from the Z to the E isomer on the ground singlet state. The black and blue curves describe the singlet (S_0) and triplet (T_1) PESs, respectively. The purely singlet pathway ($Path_{RS}$) and the triplet-channel pathway ($Path_{RT}$) are represented by the black and purple curves, respectively. The $Path_{RT}$ channel involves intersystem crossing (ISC) cascades ($S_0 \rightarrow T_1 \rightarrow S_0$).

Limitations

The computational modelling of the triplet pathway is associated with two major challenges:

1. TST theory needs to be modified. In non-adiabatic reactions such as this one, the concept of TS is replaced by the Minimum Energy Crossing Point (MECP), which is the point where the two PESs cross or become nearly degenerate. At the MECP, the **probability of hopping (psh)** is maximum, and it depends on the topology of two crossing PESs, and the strength of the Spin Orbit Coupling (SOC). In practice, psh behaves as a temperature-independent pre-factor to the TST expressions applied in adiabatic chemical reactions (i.e. Eyring equation).
2. Searching for MECP is difficult, especially in the context of screening campaigns. Constrained optimization algorithms have been designed to search for MECP, but they are currently not implemented in SCOPE.

Currently, both elements are represented by an increase of free energy (dG) associated with the triplet pathway eh psh and dE are parameters. This penalty is added through the psh value, defaulting at 0.0002

In any case, here we grouped the TS in three types:

- "**Inversion**": Referring to the inversion pathway in the **singlet** PES
- "**Rotation**": Referring to the rotation pathway in the **singlet** PES
- "**Triplet**": Referring to the rotation pathway in the **triplet** PES

Part 1. System and State subclasses

Recap from Tutorial 1...

> *A System** is a collection of chemical entities sharing common properties, typically their chemical composition.

Loading [MathJax]/extensions/Safe.js

Its main purpose is to store paths for later use in SCOPE, as well as its **sources***.

Here, we introduce the **System_azo** sub-class, tailored for azo-based molecules. In **part 4** we will show you how to create your azo system from scratch.

In [1]:
```
import os
import numpy as np
import scope
import scope_azo
```

As we did in Tutorial 1, we start by loading an existing system, called "test":

In [2]:
```
## Path of the data folder. It should be "os.path.abspath('../')+'/Data/Azo/
data_folder = os.path.abspath('../')+'/Data/9-Tutorial_9/'

## Loads the System object from a binary file, provided in the tutorial folc
sys = scope.load_binary(f"{data_folder}/Systems/test/test.npy")
```

In [3]:
```
## As usual, the Class has a __repr__ method to visualize key information
print(sys)
```

```
--------------------------------------
   >>> SCOPE System_azo Object >>>
--------------------------------------
 Name                  = test
 Dihedral Indices:     = [1, 0, 7, 8, 9, 10]
 Version               = 1.0
 Type                  = system
 Subtype               = system_azo
 Source Path           = /home/svela/SCOPE/Tutorial_azo/Sources/test/
 System File Path      = /home/svela/SCOPE/Tutorial_azo/Systems/test/
 System File Name      = /home/svela/SCOPE/Tutorial_azo/Systems/test/test.np
y
 Computations Path     = /home/svela/SCOPE/Tutorial_azo/Computations/test/

 Num Sources           = 10
     idx: type, name, formula
     0: specie, trans, H8-C12-N2-Cl2
     1: specie, cis, H8-C12-N2-Cl2
     2: specie, tsrot_a_s, H8-C12-N2-Cl2
     3: specie, tsrot_a_t, H8-C12-N2-Cl2
     4: specie, tsrot_b_s, H8-C12-N2-Cl2
     5: specie, tsrot_b_t, H8-C12-N2-Cl2
     6: specie, tsinv_l_a, H8-C12-N2-Cl2
     7: specie, tsinv_l_b, H8-C12-N2-Cl2
     8: specie, tsinv_r_a, H8-C12-N2-Cl2
     9: specie, tsinv_r_b, H8-C12-N2-Cl2

 Num Branches          = 1
     0: tutorial_azo, Num Workflows: 10
---------------------------------------------
```

Loading [MathJax]/extensions/Safe.js

```
that it follows the same structure as the core System class. Thus,
sys.set_main_path(data_folder)  # This is to set the correct path for the da
```

```
# Also, there is one additional attribute that is shown when printing the ob
# We will focus on these indices in Part 4 of this tutorial.
print(sys.dihedral_indices)
```

```
[1, 0, 7, 8, 9, 10]
```

Trans vs. Cis (Rotation about the N=N)

For simplicity, here we inspect a system for which the E and Z isomers, as well as the Transition States, have been already created. You can see how in Part 4 of this tutorial.

In [5]:
```
## These dihedral indices indicate the atoms involved in the Azo rotation. L
found, trans = sys.find_source("trans")
print(found)
trans.view(show_indices=True)
```

```
True
```

In [6]:
```
## This is an azobenzene molecule with two clorine atoms in ortho position.
# Atoms 7 and 8 are the azo itself.
# Atoms 0 and 9 are its 1st-nearest neighbours. That is, the ring atoms.
# Atoms 1 and 10 are one of the two possible 2nd-nearest neighbours of the A

## Alltogether, these 6 atoms control the main geometrical features of azobe
## These are important when generating the standard Sources of the System_az
```

In [7]:
```
## As usual, we can select its sources for inspection. Here, we select its c
found, cis = sys.find_source("cis")
print(found)

## and visualize it:
cis.view(show_indices=True)
```

```
True
```

In [8]:
```
## Notice that the azo dihedral CNNC has changed.
## To get the actual value, you can use the geometry functions implemented i
P1, P2, P3, P4 = cis.coord[cis.dihedral_indices[1:5]]      ## Store the coor
radians = scope.geometry.get_dihedral(P1, P2, P3, P4)      ## Computes the c
degrees = np.degrees(radians)                              ## Converts to de
print("CNNC cis:", degrees)

## As we will see later, this is not the optimum structure of the cis isomer
```

```
CNNC cis: 40.00000007148674
```

In [9]:
```
## If we evaluate the dihedral in the trans isomer...
P1, P2, P3, P4 = trans.coord[trans.dihedral_indices[1:5]]  ## Store the coor
radians = scope.geometry.get_dihedral(P1, P2, P3, P4)      ## Computes the c
degrees = np.degrees(radians)                              ## Converts to de
print("CNNC trans:", degrees)

## In the trans isomer, the value should approach 180º
of the bulky Cl atoms, the initial structure (generated by open-b
```

Loading [MathJax]/extensions/Safe.js

```
CNNC trans: 176.13097260926006
```

Transition States

Eight (potential) TS structures can be generated automatically with SCOPE.

In [10]:
```
## Here you can find three representative transition states, one for each pa
found, ts_inv_l_a = sys.find_source("tsinv_l_a")
found, ts_rot_a_s = sys.find_source("tsrot_a_s")
found, ts_rot_a_t = sys.find_source("tsrot_a_t")

## You can uncomment below to visualize them
#ts_inv_l_a.view(show_indices=True)
#ts_rot_a_s.view(show_indices=True)
#ts_rot_a_t.view(show_indices=True)
```

In [11]:
```
## A summary of geometry features is available in the Molecule_azo class:
ts_inv_l_a.get_geometry_summary()

## Notice the Left Inv. Angle (CNN) close to 0 degrees, corresponding to an
```

```
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):         1.2446
 Left Inv. Angle (CNN):     179.9
 Right Inv. Angle (NNC):    114.9822
 Main Dihedral (CNNC):      176.131
 Adj Dihedral Left (CCNN):  0.0
 Adj Dihedral Right (NNCC): -54.5355
```

In [12]:
```
## This is in contrast with rotation-like structures, which feature a CNNC c
ts_rot_a_s.get_geometry_summary()
```

```
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):         1.2446
 Left Inv. Angle (CNN):     113.8583
 Right Inv. Angle (NNC):    114.9822
 Main Dihedral (CNNC):      90.0
 Adj Dihedral Left (CCNN):  6.7616
 Adj Dihedral Right (NNCC): -0.0
```

In [13]:
```
## These features are also printed in the main __repr__ method of the Molecu
ts_rot_a_s
```

Loading [MathJax]/extensions/Safe.js

Out[13]:
```
----------------------------------------
------ SCOPE Molecule_azo Object ------
----------------------------------------
 Version                = 1.0
 Type                   = specie
 Sub-Type               = molecule_azo
 Name                   = tsrot_a_s
 Number of Atoms        = 24
 Formula                = H8-C12-N2-Cl2
 Charge                 = 0
 Spin (alpha - beta)    = 0
 Number of Parents      = 0
 Has Adjacency Matrix   = NO
 Has Bonds              = NO
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):          1.2446
 Left Inv. Angle (CNN):      113.8583
 Right Inv. Angle (NNC):     114.9822
 Main Dihedral (CNNC):       90.0
 Adj Dihedral Left (CCNN):   6.7616
 Adj Dihedral Right (NNCC): -0.0
```

In [14]:
```
## Notice that tsrot_a_s is defined as a singlet (spin = 0). The same holds
print("Spin:", ts_rot_a_s.spin)
```

```
Spin: 0
```

In [15]:
```
## Instead, the 'triplet' TSs are defined as spin = 2 (as expected)
print(ts_rot_a_t)
```

```
----------------------------------------
------ SCOPE Molecule_azo Object ------
----------------------------------------
 Version                = 1.0
 Type                   = specie
 Sub-Type               = molecule_azo
 Name                   = tsrot_a_t
 Number of Atoms        = 24
 Formula                = H8-C12-N2-Cl2
 Charge                 = 0
 Spin (alpha - beta)    = 2
 Number of Parents      = 0
 Has Adjacency Matrix   = NO
 Has Bonds              = NO
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):          1.2446
 Left Inv. Angle (CNN):      113.8583
 Right Inv. Angle (NNC):     114.9822
 Main Dihedral (CNNC):       90.0
 Adj Dihedral Left (CCNN):   6.7616
 Adj Dihedral Right (NNCC): -0.0
```

Loading [MathJax]/extensions/Safe.js

Part 2: Thermal Properties

Trans vs. Cis stability

In [16]:
```
## From the results, one can easily retrieve the Cis - Trans energy differen
sys.get_thermal_stability(target_state='opt', temp=298.15, debug=0)

## The 'target_state' variable serves to indicate the name of the state from
## Here, we used 'opt', and must be the same name for both isomers.
```

Out[16]:
```
dG_cis-trans:   3.55011349 kcal/mol
```

Finding the Minimum Energy Transition State (METS)

In [17]:
```
# To find the METS, the function get_mets() goes through all sources with th
# - For sources defined in a singlet state, it verifies that it is a TS (one
# - For sources defined as a triplet, it verifies that it is a minimum (all
#   Also, it applies an energy penalty to Gtot, to compensate for the ISC pr

#   This penalty is applied as dE = - RT ln(p_sh)
#    - where p_sh is the probability of surface hopping.
#   By default, p_sh is set to 0.0002, so dE amounts 5 kcal/mol at 298.15 K.
#   This value is a parameter, and can be modified by the user.

## Finally, it selects and returns the source with a lowest Gtot at the requ
mets = sys.get_mets(temp=298.15, target_state='opt', force=True, p_sh=0.0002
```

In [18]:
```
## In this case, the METS is a structure associated with a rotation in the t
print(mets)
```

Loading [MathJax]/extensions/Safe.js

```
---------------------------------------
------ SCOPE Molecule_azo Object ------
---------------------------------------
 Version               = 1.0
 Type                  = specie
 Sub-Type              = molecule_azo
 Name                  = tsrot_a_t
 Number of Atoms       = 24
 Formula               = H8-C12-N2-Cl2
 Charge                = 0
 Spin (alpha - beta)   = 2
 Number of Parents     = 0
 Has Adjacency Matrix  = NO
 Has Bonds             = NO
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):         1.2446
 Left Inv. Angle (CNN):     113.8583
 Right Inv. Angle (NNC):    114.9822
 Main Dihedral (CNNC):      90.0
 Adj Dihedral Left (CCNN):  6.7616
 Adj Dihedral Right (NNCC): -0.0
```

In [19]:

```
## The Geometry Features shown above refer to the SOURCE in its INITIAL stat
found, mets_opt_state = mets.find_state("opt")
print(mets_opt_state)
```

Loading [MathJax]/extensions/Safe.js

```
-----------------------------------
------ SCOPE State_azo Object ------
-----------------------------------
 Name                  = opt
 Source Name           = tsrot_a_t
 Source Type           = specie
 Labels                = C...
 Coord                 = [-0.173125, -1.044504, -0.658308]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 19.8729

 Num of Molecules:     = 1
 With Formulae:
    0: H8-C12-N2-Cl2
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):         1.2661
 Left Inv. Angle (CNN):     127.971
 Right Inv. Angle (NNC):    128.0156
 Main Dihedral (CNNC):      110.9285
 Adj Dihedral Left (CCNN):  -11.9885
 Adj Dihedral Right (NNCC): -11.8401
```

Gtot vs. Gtot_eff

In [20]:
```
## Beyond geometry, this state is associated with a Gtot value at 298.15K.
## This value includes vibrational and electronic enthalpy and entropy terms
## This are automatically computed during sys.get_mets(), and stored appropr
Gtot_eff = mets_opt_state.results['Gtot_eff']

## Technically, Gtot is an instance of the Collection-class:
print(Gtot_eff)
```

```
--------------------------------------------------
   COLLECTION OF DATA   = Gtot_eff
--------------------------------------------------
 Variable              = temperature
 Num of Entries        = 1
 First                 = -1490.8317518596448
 Last                  = -1490.8317518596448
 Units                 = au
 Function              = state_azo.get_gtot_eff()
```

In [21]:
```
## Collections store multiple values associated with the same variable (here
## To find the Gtot value at a specific temperature, you can do:
print(Gtot_eff.find_value_with_property('temperature', 298.15))
```

```
Gtot_eff: -1490.83175186 au (at temperature=298.15)
```

Loading [MathJax]/extensions/Safe.js

In [22]:
```
# The name 'Gtot_eff' means that any energy penalty has already been conside
# In this case, since the METS is a triplet, Gtot_eff differs from the 'orig
Gtot = mets_opt_state.results['Gtot'].find_value_with_property('temperature'
print(Gtot)
## Notice the slight difference in energy...
```

```
Gtot: -1490.83979360 au (at temperature=298.15)
```

In [23]:
```
## This difference in energy corresponds to the energy penalty associated wi
diff = Gtot - Gtot_eff.find_value_with_property('temperature', 298.15)
diff.convert_to_units('kcal')

## In singlet states, Gtot and Gtot_eff should be equal
```

Out[23]:
```
sum_Gtot:  -5.04627077 kcal
```

Halflife time

In [24]:
```
## Overall, with the METS energy, and that of the Trans isomer, one can get
sys.get_cis_halflife_time(temp=298.15, debug=0)
```

Out[24]:
```
cis_halflife_time: 264.61534987 s (at temperature=298.15)
```

In [25]:
```
# The halflife time is stored as a result for this system:
print(sys.results['cis_halflife_time'])

# Together with the thermal relaxation rate
print(sys.results['rate_thermal_cis2trans'])

# and the dG=(G_TS - G_cis)
print(sys.results['dG_from_cis'])
```

```
cis_halflife_time: 264.61534987 s (at temperature=298.15)
rate_thermal_cis2trans:   0.00261945 s^-1 (at temperature=298.15)
dG_from_cis:  20.97520978 kcal/mol (at temperature=298.15)
```

In [26]:
```
## This is also possible for the trans isomer:
sys.get_trans_halflife_time(temp=298.15, debug=0)
```

Out[26]:
```
trans_halflife_time: 105897.54166269 s (at temperature=298.15)
```

Part 3. Optical Properties

In [27]:
```
# Apart from the cis-trans thermal stability, one of the key properties of A
# For the test compound, the absorption spectrum is simulated using the exci
# Here, this data is stored in the 'opt' state.
found, trans_source = sys.find_source('trans')
found, trans_state  = trans_source.find_state('opt')
print(trans_state)
```

Loading [MathJax]/extensions/Safe.js

```
-----------------------------------
------ SCOPE State_azo Object ------
-----------------------------------
 Name                  = opt
 Source Name           = trans
 Source Type           = specie
 Labels                = C...
 Coord                 = [-0.031101, -0.036525, 0.03185]...
 Number of Units (Z)   = 1

 Has VNMs              = YES
 Is Minimum            = True
 Is a Transition State = False
 First Freq (cm-1)     = 17.3076

 Has Excited States    = YES
 Num of Molecules:     = 1
 With Formulae:
    0: H8-C12-N2-Cl2
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):         1.2372
 Left Inv. Angle (CNN):     115.3575
 Right Inv. Angle (NNC):    115.3682
 Main Dihedral (CNNC):      179.9971
 Adj Dihedral Left (CCNN):  42.0511
 Adj Dihedral Right (NNCC): -42.1224
```

In [28]:

```
# The excited states are stored as an attribute:
trans_state.exc_states
```

Loading [MathJax]/extensions/Safe.js

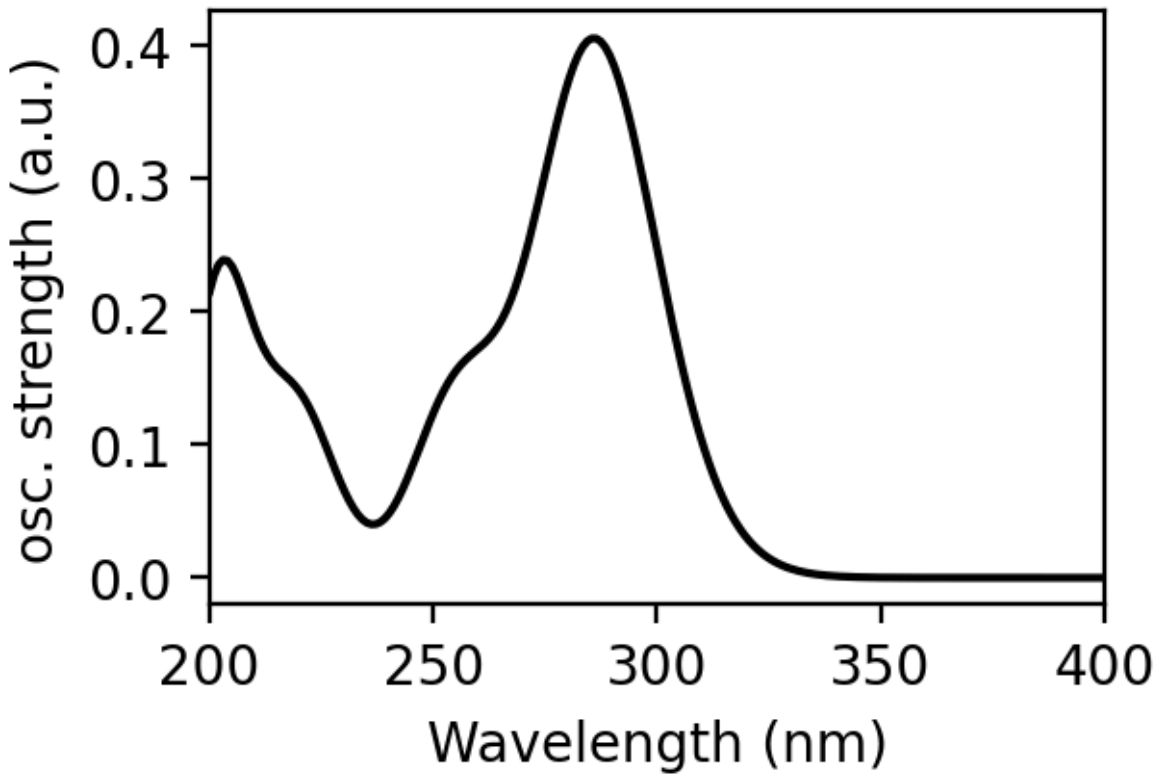
Out[28]:
```
[-------------------------------------------
 Index                  = 1
 Energy (eV)            = 2.6097 eV
 Wavelength (nm)        = 475.08 nm
 Oscillator Strength    = 0.0,
-------------------------------------------
 Index                  = 2
 Energy (eV)            = 4.3288 eV
 Wavelength (nm)        = 286.42 nm
 Oscillator Strength    = 0.3999,
-------------------------------------------
 Index                  = 3
 Energy (eV)            = 4.6914 eV
 Wavelength (nm)        = 264.28 nm
 Oscillator Strength    = 0.0,
-------------------------------------------
 Index                  = 4
 Energy (eV)            = 4.8376 eV
 Wavelength (nm)        = 256.29 nm
 Oscillator Strength    = 0.1448,
-------------------------------------------
 Index                  = 5
 Energy (eV)            = 5.4584 eV
 Wavelength (nm)        = 227.14 nm
 Oscillator Strength    = 0.0,
-------------------------------------------
 Index                  = 6
 Energy (eV)            = 5.5735 eV
 Wavelength (nm)        = 222.45 nm
 Oscillator Strength    = 0.0698,
-------------------------------------------
 Index                  = 7
 Energy (eV)            = 5.6879 eV
 Wavelength (nm)        = 217.98 nm
 Oscillator Strength    = 0.0,
-------------------------------------------
 Index                  = 8
 Energy (eV)            = 5.7268 eV
 Wavelength (nm)        = 216.5 nm
 Oscillator Strength    = 0.0693,
-------------------------------------------
 Index                  = 9
 Energy (eV)            = 6.0569 eV
 Wavelength (nm)        = 204.7 nm
 Oscillator Strength    = 0.0,
-------------------------------------------
 Index                  = 10
 Energy (eV)            = 6.1227 eV
 Wavelength (nm)        = 202.5 nm
 Oscillator Strength    = 0.2264]
```

Absorption Spectrum

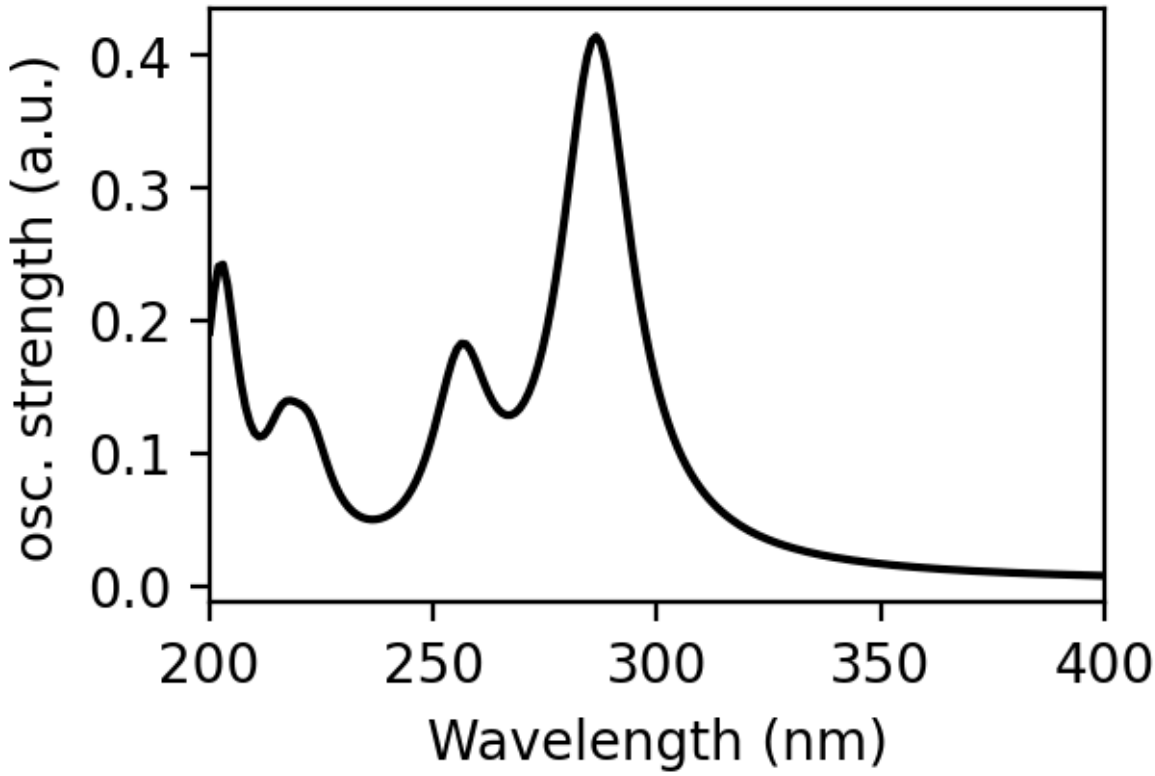
Loading [MathJax]/extensions/Safe.js

```
her, they generate the absorption spectrum
trans_state.plot_abs_spectrum(lmin=200, lmax=400, function='gaussian', sigma
```

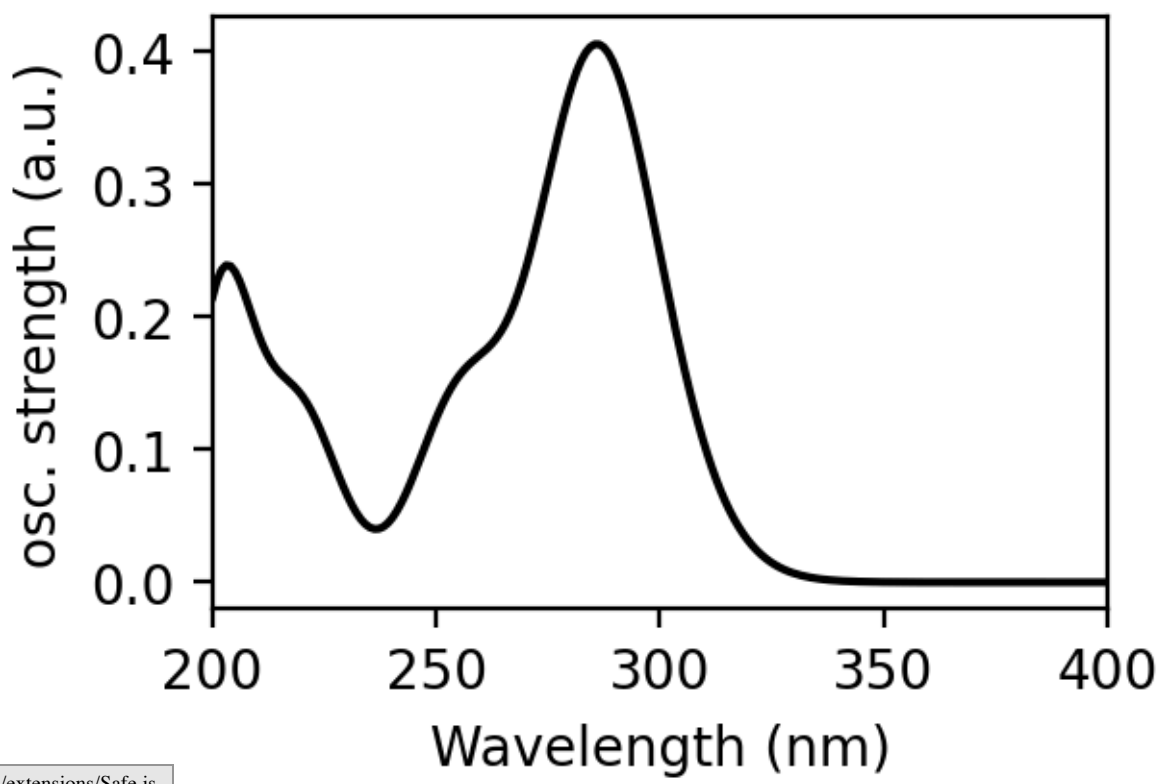
```
# Alternatively, if you prefer the spectrum as x, y arrays, you can run:
# x, y = trans_state.get_abs_spectrum(lmin=200, lmax=400, function='gaussian

# In both cases, we selected gaussian functions with sigma=0.2 for the convo
# Lorentzian and Laplacian functions can be used instead, as well as other s
trans_state.plot_abs_spectrum(lmin=200, lmax=400, function='lorentzian', sig
```



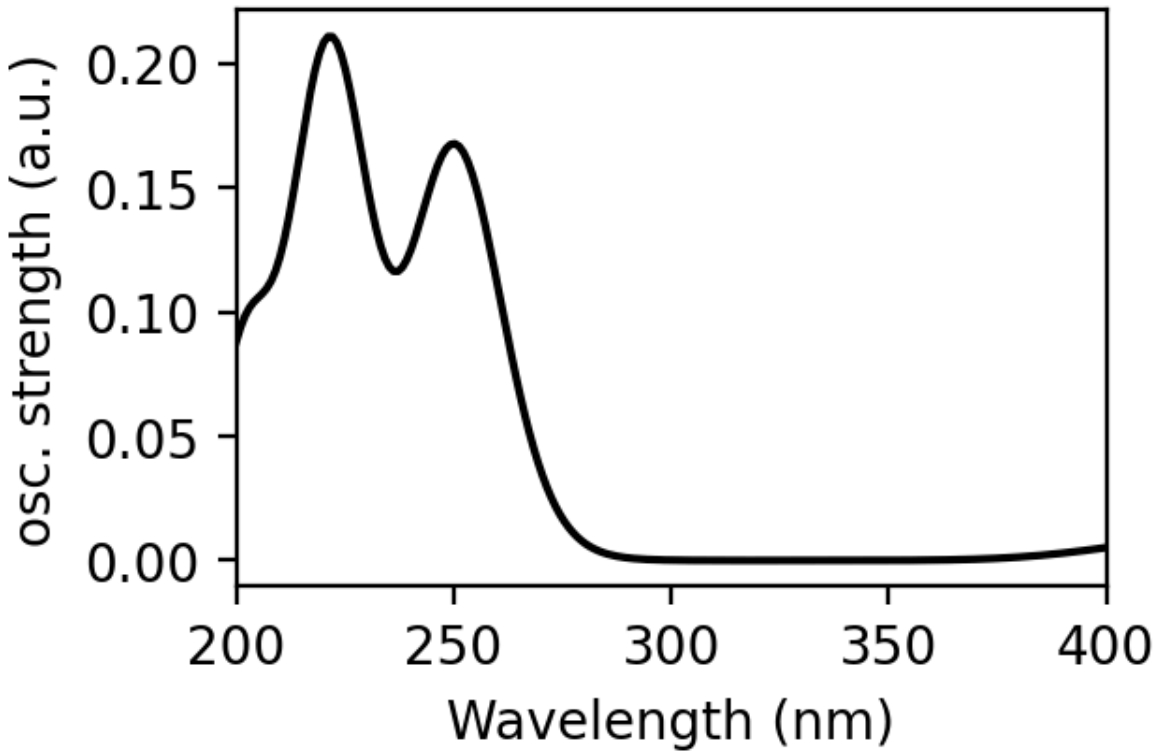
Loading [MathJax]/extensions/Safe.js



In [30]:
```
# Of course, the same holds for the 'cis' isomer:
found, cis_source = sys.find_source('cis')
found, cis_state  = cis_source.find_state('opt')
trans_state.plot_abs_spectrum(lmin=200, lmax=400, function='gaussian', sigma
cis_state.plot_abs_spectrum(lmin=200, lmax=400, function='gaussian', sigma=0
```



Loading [MathJax]/extensions/Safe.js



The Photo-Stationary State (PSS)

Under light irradiation, azo molecules reach a **photostationary state (PSS)** in which both the trans (E) and cis (Z) isomers are populated. The final composition depends on the competition between **thermal relaxation** and **photo-induced isomerization** processes.

Starting from the steady-state condition:

$$ \displaystyle \frac{d[E]}{dt} = - (k^{ph}_{EZ} + k^{th}_{EZ}) [E] + (k^{ph}_{ZE} + k^{th}_{ZE}) [Z] = 0 $$
Then,

$$ \displaystyle k_{EZ}[E] = k_{ZE}[Z] $$
Within this framework, the fraction -population- of the E isomer under irradiation (N_E) can be written as:

$$ \displaystyle N_E = \frac{k_{ZE}}{k_{EZ} + k_{ZE}} =\frac{k^{th}_{ZE} + k^{ph}_{ZE}}{k^{th}_{EZ} + k^{ph}_{EZ} + k^{th}_{ZE} + k^{ph}_{ZE}}; \quad $$$ N_Z = 1- N_E $$ where k^{th}_{EZ} and k^{th}_{ZE} elements are the **thermal rate constants** for the conversion from trans to cis (EZ) and cis to trans (ZE) while k_{EZ}^{ph} and k_{ZE}^{ph} are the **photoisomerization rate constants**.

The **thermal rates k^{th}_{EZ} and k^{th}_{ZE}** are normally evaluated using

Loading [MathJax]/extensions/Safe.js ion, or obtained from experimental half-life times, $t_{1/2}$ assuming a

first-order reaction:

$$k^{th} = \frac{ln(2)}{t_{1/2}} \quad [s^{-1}]$$

The **photochemical rates k^{ph}_{EZ} and k^{ph}_{ZE}** can be modeled as

$$ \displaystyle k^{ph}_{EZ} = \phi_{EZ} \int \sigma_E(\lambda)\,\Phi(\lambda)\, d\lambda \quad [s^{-1}] $$
by combining three physical quantities:

- the **absorption probability** of the molecule (or absorption cross section): $\sigma_E(\lambda)$ [m^2]
- the **number of incoming photons** (or photon flux): $\Phi(\lambda)$ [$photons·m^{-2} s^{-1} nm^{-1}$]
- the **probability that absorption leads to reaction** (or quantum yield): ϕ_{EZ} [$a.u$]

More specifically:

- $\Phi(\lambda)$ represents the number of photons reaching the sample per unit area, per unit time, and per wavelength interval. It is obtained from the irradiance divided by the energy of a single photon:

$$ \Phi(\lambda) = \frac{I(\lambda)}{E_{photon}}; \quad \frac{\left[\frac{J}{s·m^2 nm^{-1}}\right]}{\left[J \right]} $$

- $\sigma_E(\lambda)$ quantifies the probability that a molecule absorbs a photon of wavelength λ. **Note:** *The spectra obtained by using get_abs_spectrum() returns the oscillator strengths vs. wavelength, $f(\lambda)$. To provide physical meaning (by converting it to an absorption cross section), the y-axis is converted to* m^2 *by applying a factor K (note that a s^{-1} should appear, and it is cancelled by frequency units implicit in the wavelength space):*

$$ \sigma_E(\lambda) = \frac{\pi · h · e}{ \varepsilon _0 · m_e · c } * f(\lambda) \quad \left[m^2 \right] $$

- ϕ_{ZE} measures the efficiency of the photochemical reaction. It corresponds to the fraction of absorbed photons that result in isomerization (for example, $\phi_{ZE}=0.5$ means that half of the absorbed photons lead to reaction).

In practice, light sources are not perfectly monochromatic (even lasers emit photons across a finite wavelength range). For this reason, contributions from all wavelengths must be considered, which is achieved through the integral above. The resulting equation corresponds to the **total absorption rate per molecule** and can be interpreted as a spectral overlap between the lamp emission spectrum and the molecular absorption spectrum.

Loading [MathJax]/extensions/Safe.js

Once the populations in the steady state, and N_E,N_Z at an given irradiation wavelength are known, it is possible to obtain PSS absorption cross section spectrum $$ \displaystyle \sigma_{PSS}(\lambda) = N_E \sigma_{E}(\lambda)+ N_Z \sigma_{Z}(\lambda) $$

along with the molar absorptivity, ε in $L · mol^{-1}·cm^{-1}$, applying the Lambert-Beer concentration factors

$$ \displaystyle \varepsilon_{PSS}(\lambda) = \frac{N_A}{1000 · ln(10)} \sigma_{PSS}(\lambda) $$
Then, the molar absorptivity at the steady state can be represented graphically for each irradiation wavelength $\displaystyle \varepsilon_{PSS}(\lambda)$ vs. λ.

The 'Lamp' Class

In [31]:
```
from scope_azo.azo_classes import Lamp
```

In [32]:
```
# The properties of the light source are provided to the PSS as a Lamp-Class
# Lamps store information about the available wavelengths, and associated FW
# If you don't have such information, you can use a "default" Lamp that we p
# Any Lamp can be tuned at will by the user, and new Lamps can also be added
my_lamp = Lamp("default")
print(my_lamp)
```

```
--------- Scope Lamp Object -----------
 Name             = default
 Wavelengths      = [  0 250 280 310 340 370 400 430 460 490 520 550 580] nm
 FWHM             = [ 1. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20.] nm
 Power            = [ 0. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50.] mW
 Power Intensity  = 1.0
----------------------------------------
```

In [33]:
```
# Basically, this Lamp provides 12 excitation wavelentghs defined as:
# - gaussians of FWHM=20 nm
# - with 50 mW of power when used at 100% (Power Intensity=1.0). Can be char
```

In [34]:
```
# For the moment, only one real-world device has been included, the CoolLED
my_lamp = scope_azo.azo_classes.Lamp("coolled")
print(my_lamp)
```

Loading [MathJax]/extensions/Safe.js

```
--------- Scope Lamp Object -----------
 Name             = coolled
 Wavelengths      = [  0 365 385 405 435 460 470 500 525 550 595 635 660 740
770] nm
 FWHM             = [ 1.   12.54 11.38 16.08 13.02 17.61 23.26 24.72 28.32 8
2.79 14.61 16.72
 21.42 41.12 27.75] nm
 Power            = [  0.    15.12  59.39 188.32 138.75 240.19 192.29  59.26
39.   166.74
  32.52 123.88 159.01  36.62  15.61] mW
 Power Intensity  = 1.0
----------------------------------------
```

In [35]:

```python
# Finally, absence of irradiation can be simulated with the "dark" Lamp:
my_lamp = scope_azo.azo_classes.Lamp("dark")
print(my_lamp)
```

```
--------- Scope Lamp Object -----------
 Name             = dark
 Wavelengths      = [0] nm
 FWHM             = [1.] nm
 Power            = [0.] mW
 Power Intensity  = 0.0
----------------------------------------
```

In [36]:

```python
# To compute the PSS, one just needs to use the get_PSS() function as:
my_pss = sys.get_PSS(lamp_name='default', target_state='opt', temp=298.15, l

# This function has several options and defaults, do not forget to read the
#help(sys.get_PSS)

# In summary, the function takes/computes all relevant data from the stored
print(my_pss)

# This class stores relevant data to compute the PSS at any given wavelength
```

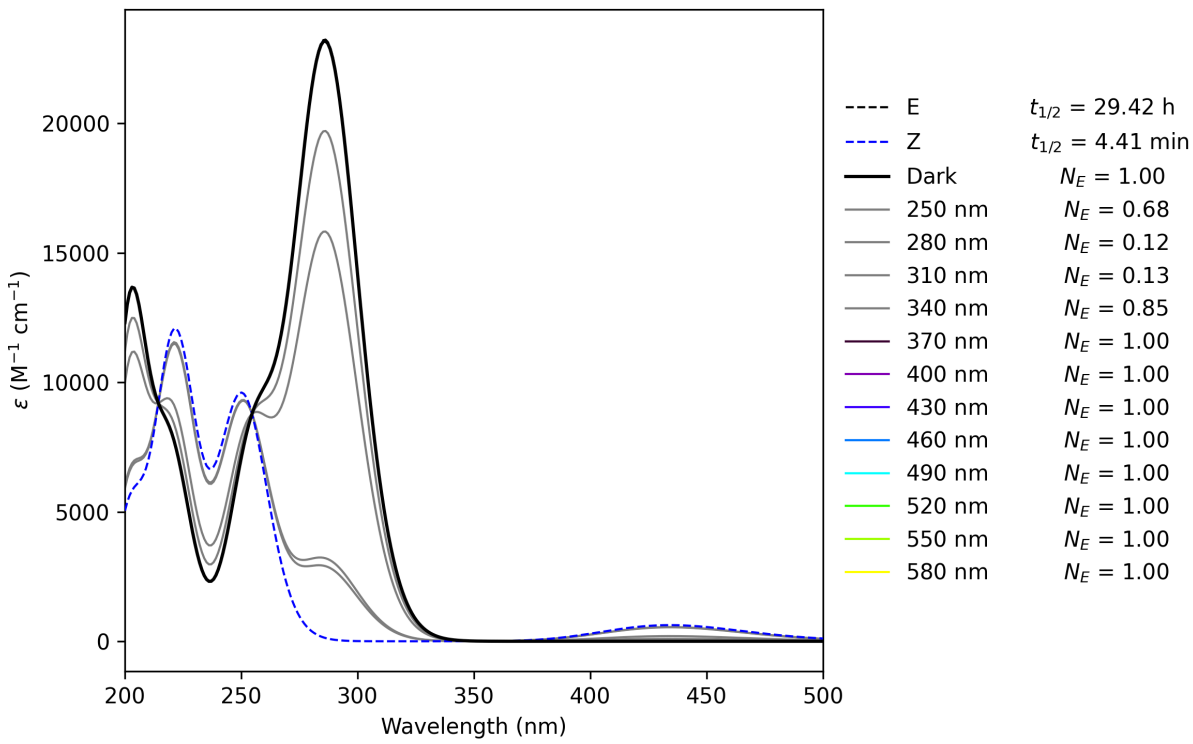
```
------------------------------
   >>> SCOPE PSS Object >>>
------------------------------
 System             = test
 Lamp Name          = default
 Lamp Wavelengths   = [  0 250 280 310 340 370 400 430 460 490 520 550 580]
(nm)
 Lamp FWHM          = [ 1. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20. 20.]
(nm)
 Lamp Powers        = [ 0. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50. 50.]
(mW)
 Illuminated Area   = 78.539816 (mm2)
 Rate Thermal E->Z  = 6.545451e-06 (s^-1)
 Rate Thermal Z->E  = 2.619452e-03 (s^-1)
 Num of Results     = 13
```

Loading [MathJax]/extensions/Safe.js

In [37]:
```
# Wavelength-dependent results are stored as a dictionary, with wavelengths
res = my_pss.pss_results[250]

print(res['wavelength'])                ## Wavelength of irradiation
print(res['pss_trans_ratio'])           ## Ratio of Trans Isomer in the PSS
print(res['rate_photo_trans2cis'])      ## Photoisomerization Rate for E-->Z
print(res['rate_photo_cis2trans'])      ## Photoisomerization Rate for Z-->E
print(res['rate_thermal_trans2cis'])    ## Thermal Reaction Rate for E-->Z (
print(res['rate_thermal_cis2trans'])    ## Thermal Reaction Rate for Z-->E (
print(res['QY_trans2cis'])              ## Quantum Yield for E-->Z photoisom
print(res['QY_cis2trans'])              ## Quantum Yield for Z-->E photoisom
```

```
250
0.6792281181392162
0.01352218580838394
0.026027369078849585
6.5454510999680045e-06
0.0026194518983660817
0.3
0.5
```

In [38]:
```
# Results can be plotted with:
my_pss.plot_spectra(lmax=500)
```



In [ ]:
```
# Above, we can see that we only obtain a mixture of E and Z isomers under l
# For all other cases, only the trans state is present

# These are, unfortunately, boring results from the physico-chemical point c
```

Part 4. Creating new Azo-based Systems

Loading [MathJax]/extensions/Safe.js

Here we show how to create a new system from scratch, and feed it with **sources**. All steps should proceed seamlessly if you follow the instructions shown below.

In [40]:
```
# First, let's define a SMILES string. It is important to be careful when
#   constructing your SMILES: it MUST follow the pattern below to correctly
#   atom indices near the Azo group, enabling rotation and inversion of both

#   IMPORTANT: SMILES strings MUST adhere to the following structure:
#                    'C1(...C1)/N=N/ring2'
#                              or
#                    'C1(...C1)/N=N\\ring2'
#
#              That is, it doesn't matter if the SMILES string is in trans (
#              However, there is a specific way to define both rings. See be

# We will select azobenzene for its simplicity. Let's store the SMILES strin

smiles  = 'c1(ccccc1)/N=N\\c2ccccc2'
name    = 'Azobenzene'
```

In [41]:
```
# We will create our system as a System_azo object providing its name and th
new_sys = scope_azo.System_azo(name, smiles)
print(new_sys)
```

```
-------------------------------------
   >>> SCOPE System_azo Object >>>
-------------------------------------
 Name                  = Azobenzene
 Dihedral Indices:     = [1, 0, 6, 7, 8, 9]
 Version               = 1.0
 Type                  = system
 Subtype               = system_azo
----------------------------------------------
```

Creating E, Z and TS

In [42]:
```
# System_azo.create_trans()
    # Creates trans structures.
    # It uses the SMILES string stored in the System to generate 3D coordina
    # If everything works, it adds the trans isomer as a `Molecule_azo` obje
    # Returns the `Molecule_azo` object.
```

In [43]:
```
# Let's create the trans isomer. It is automatically registered as a source
# Note that the 'overwrite' parameter defaults to False.
trans = new_sys.create_trans(overwrite=True)
print(trans)

# If Openbabel is not installed, it will raise and error. Install it with 'c
```

Loading [MathJax]/extensions/Safe.js

```
----------------------------------------
------ SCOPE Molecule_azo Object ------
----------------------------------------
 Version               = 1.0
 Type                  = specie
 Sub-Type              = molecule_azo
 Name                  = trans
 Number of Atoms       = 24
 Formula               = H10-C12-N2
 Charge                = 0
 Spin (alpha - beta)   = 0
 SMILES                = c1(ccccc1)/N=N/c2ccccc2
 Number of Parents     = 0
 Has Adjacency Matrix  = NO
 Has Bonds             = NO
-------------------------------------------------
 Main Geometry Features of the Azo group:
-------------------------------------------------
 NN distance (dNN):         1.251
 Left Inv. Angle (CNN):     112.6651
 Right Inv. Angle (NNC):    114.0996
 Main Dihedral (CNNC):      173.1404
 Adj Dihedral Left (CCNN):  -178.177
 Adj Dihedral Right (NNCC): 143.5666
```

In [44]:

```
# System_azo.create_cis(target_deg=40.)
    # Creates cis structures from the trans isomer. if the trans isomer is r
    # If everything works, it adds the cis isomer as source of the system.
    # The `target_deg` parameter allows you to choose the torsion of the dih
    # By default, it is set to 40.0 degrees.
cis = new_sys.create_cis(target_deg=30.)
print(cis)
```

Loading [MathJax]/extensions/Safe.js

```
----------------------------------------
------ SCOPE Molecule_azo Object ------
----------------------------------------
 Version               = 1.0
 Type                  = specie
 Sub-Type              = molecule_azo
 Name                  = cis
 Number of Atoms       = 24
 Formula               = H10-C12-N2
 Charge                = 0
 Spin (alpha - beta)   = 0
 SMILES                = c1(ccccc1)/N=N\c2ccccc2
 Number of Parents     = 0
 Has Adjacency Matrix  = NO
 Has Bonds             = NO
-------------------------------------------------
 Main Geometry Features of the Azo group:
-------------------------------------------------
 NN distance (dNN):         1.251
 Left Inv. Angle (CNN):     112.6651
 Right Inv. Angle (NNC):    114.0996
 Main Dihedral (CNNC):      30.0
 Adj Dihedral Left (CCNN):  -151.0
 Adj Dihedral Right (NNCC): 168.0
```

Creation of transition states

In [45]:
```
# System_azo.create_ts(ts_list=['tsrot', 'tsinv', 'triplet'])
#
#   The `ts_list` parameter allows you to specify which (TS) structures will
#
# - tsrot: Generates the rotation of the azo group. It creates two
#   structures in the singlet state with the central C-N=N-C dihedral
#   angle set to +90 deg (TSrot_a_s) and -90 deg (TSrot_b_s).
#
# - tsinv_l & tsinv_r: Generates the left (tsinv_l) and right (tsinv_r) inve
#     * The angle of the inverting nitrogen (C-N=N for the left
#       ring, or N=N-C for the right ring) is changed to 180 deg.
#     * Each type of inversion (r and l) creates two variants (a and b),
#       indicating that the adjacent dihedral is set to 0 and 180 deg, respe
#
# - triplet: If included, this flag also generates the rotational
#   transition states (tsrot) in the triplet state (spin = 2)
#
# By default, all are generated
```

In [46]:
```
# Following the naming convention above:
ts_list = new_sys.create_ts()
print([ts.name for ts in ts_list])
```

```
['tsrot_a_s', 'tsrot_a_t', 'tsrot_b_s', 'tsrot_b_t', 'tsinv_l_a', 'tsinv_l_
b', 'tsinv_r_a', 'tsinv_r_b']
```

Loading [MathJax]/extensions/Safe.js

In [47]:
```
# And we can check if geometries are correct using the __repr__ method inclu
ts = new_sys.find_source('tsrot_a_s')[1]
print(ts)
# Check the Azo group dihedral angle is set to ca. 90 degree
```

```
----------------------------------------
------ SCOPE Molecule_azo Object ------
----------------------------------------
 Version                = 1.0
 Type                   = specie
 Sub-Type               = molecule_azo
 Name                   = tsrot_a_s
 Number of Atoms        = 24
 Formula                = H10-C12-N2
 Charge                 = 0
 Spin (alpha - beta)    = 0
 Number of Parents      = 0
 Has Adjacency Matrix   = NO
 Has Bonds              = NO
--------------------------------------------------
 Main Geometry Features of the Azo group:
--------------------------------------------------
 NN distance (dNN):          1.251
 Left Inv. Angle (CNN):      112.6651
 Right Inv. Angle (NNC):     114.0996
 Main Dihedral (CNNC):       90.0
 Adj Dihedral Left (CCNN):   -178.177
 Adj Dihedral Right (NNCC): 0.0
```

Careful with Steric Hindrance: set_dihedral() vs. solve_dihedral()

In [48]:
```
# Remember the indices of the atoms involved in the Azo dihedral, accessible
print('Atom Indices for Dihedral: ', new_sys.dihedral_indices)

# The atoms involved in the C-N=N-C rotation correspond to indices 0, 6, 7,
# The first and last elements of this list correspond to the anchor atoms of
# left and right rings, respectively, and are used to facilitate independent
atom0, atom1, atom2, atom3, atom4, atom5 = new_sys.dihedral_indices
```

```
Atom Indices for Dihedral:  [1, 0, 6, 7, 8, 9]
```

In [49]:
```
# Using the core SCOPE module, modifying dihedral angles should be straightf
rotated_coord       = scope.geometry.set_dihedral(trans.labels, trans.coord,
rotated_molec       = scope_azo.Molecule_azo(trans.labels, rotated_coord)
rotated_molec.view(show_indices=True)

# However, steric clashes might appear. Se atoms 5, 13, 18, and 23 below.
```

In [50]:
```
# Instead of using the function set_dihedral() you can use solve_dihedral()
# Both functions are in scope.geometry()

lved_coord       = scope.geometry.solve_dihedral(trans.labels, rot
```

Loading [MathJax]/extensions/Safe.js

```
# rotated_coord are the coordinates with steric hindrance, obtained with set
# solved_coord are the coordinates without steric hindrance, obtained with s
solved_molec = scope_azo.Molecule_azo(trans.labels, solved_coord)
solved_molec.view(show_indices=True)
```

In [51]:
```
# In essence, solve_dihedral() rotates the adjacent dihedral angles until th
# This only works if the 'central', and two adjacent dihedrals are rotatable
# If we give other atom indices:
worked, wont_work_coord    = scope.geometry.solve_dihedral(trans.labels, rot

# You should receive a message complaining that the dihedrals are not rotata
# That is, a check is performed rather than just rotating blindly.
# This is basically what create_cis() and create_ts() functions do to avoid
```

```
SOLVE_DIHEDRAL: Adjacent dihedrals are not rotatable. Returning original coo
rdinates.
```

> **Pro Tip:** We recommend using `solve_dihedral()` for geometries prone to steric hindrance, if possible. It automatically adjusts adjacent rings to ensure a valid, clash-free conformation.

Command Line Interface (CLI)

To create a System_azo using the CLI, and access the optical and thermal properties, you can do the following:

1. Create and Configure the Environment:

```
scope config -n name
```

2. Create the System (alternatively, you can create it here and upload it)

```
scope_azo create_single -n path_to_environment_file -s azobenzene -
smi 'c1(ccccc1)/N=N\\c2ccccc2' -f -ts
```

3. Run the computations with SCOPE using the input files (.scope) provided in /Data/9-Tutorial_9/

```
scope run -n path_to_environment_file -s azobenzene -i 1-
opt_min.scope  2-opt_ts.scope  3-freq_min.scope  4-freq_ts.scope
5-td_min.scope -v -e
```

Loading [MathJax]/extensions/Safe.js